

VNRVJiet

CREATING MICROSERVICES IN JAVA

Student Mark Service For DevOps Integration



Table of Contents

Prerequisite Installations:	3
Introduction:	4
Step 1: Creating Java Spring Boot Project Template Using Spring Initializer	4
Configuring The Project Setting	4
Adding Dependency Jars	5
Generating the Project Template	5
Extracting the Project Template ZIP Locally	5
Step 2: Making The Project Template Ready For Coding Microservices	6
Understanding Project Template Folder Structure	6
Understanding Important Content Of pom.xml File	7
Configure Application Properties	8
Step 3: Add Source Code	9
<i>StudentMarkServiceApplication Class</i>	9
<i>Student Class</i>	9
<i>StudentMarkServiceController Class</i>	10
<i>StudentMarkService Class</i>	12
<i>StudentMarkServiceRepository Class</i>	13
Step 4: Running DevOps process	14
<i>DevOps Process</i>	14
<i>Jenkinsfile</i>	15
<i>Dockerfile</i>	17
<i>namespace-template.yaml</i>	19
<i>deployment-template.yaml</i>	20
<i>service-template.yaml</i>	22

Date	Changes	Documentor
28 th October 2025	First draft version	B.S.Satyadev
31 st October 2025	Updated with full code images	B.S.Satyadev
02 nd November 2025	Final Document	B.S.Satyadev

Prerequisite Installations:

Before using this guide following applications have to be installed in the target machine. It is assumed that the target machine is a Windows machine.

1. [JDK 21 For Windows](#)
2. [Git for Windows](#)
3. [Create GitHub Account](#)
4. [IntelliJ IDEA Community Edition IDE](#)
5. [Jenkins](#)
6. [ngrok](#)
7. [Docker Desktop \(With K8s\)](#)
8. [Postman](#)

Introduction:

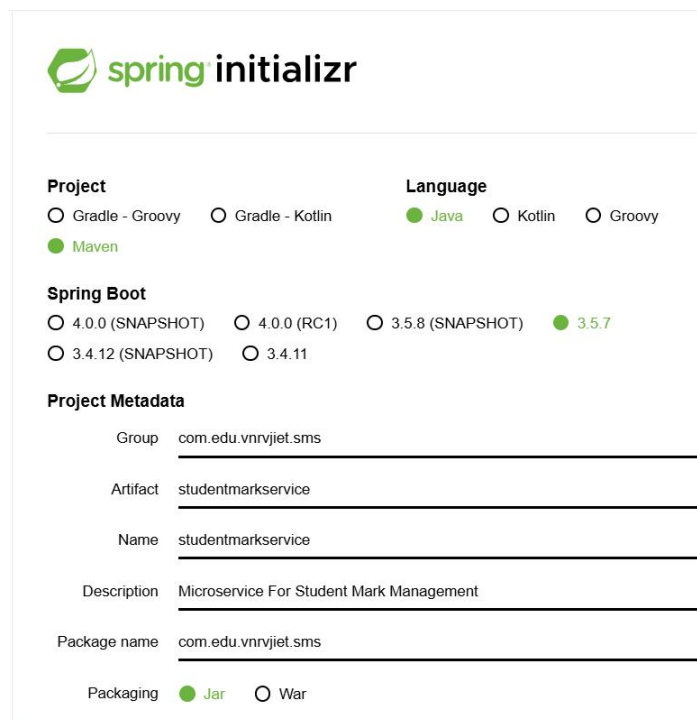
This document is a **step by step guide** for creating **RE**presentational **State** **T**ransfer (**REST**) architecture over Hyper Text Transfer Protocol (HTTP) style based **Java Microservice** Application that is built upon Spring Boot Framework, using **IntelliJ IDEA Community Edition Integrated Development Environment (IDE)**. As a means to explain the creation of REST/HTTP based Microservice, **Student Marks Service** project is considered. Microservice in the this project demonstrates HTTP methods: **GET, POST, DELETE &, PUT**

Step 1: Creating Java Spring Boot Project Template Using Spring Initializer

Configuring The Project Setting

One caveat with **IntelliJ IDEA Community Edition IDE** is that it doesn't provide **Project Template** for creating Java Spring Boot Framework based applications, unless the IDE is upgraded to **Ultimate Edition**. So a work around for creating **Project Template** is to use **Spring Initializer Web Site** :

1. Go to <https://start.spring.io/>
2. Set following configurations in the site:



The screenshot shows the Spring Initializer web form with the following configurations:

- Project:** ☒ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ **Java**, ☐ Kotlin, ☐ Groovy
- ☒ **Maven**
- Spring Boot:** ☐ 4.0.0 (SNAPSHOT), ☐ 4.0.0 (RC1), ☐ 3.5.8 (SNAPSHOT), ☒ **3.5.7**, ☐ 3.4.12 (SNAPSHOT), ☐ 3.4.11
- Project Metadata:**
 - Group: com.edu.vnnvjiet.sms
 - Artifact: studentmarkservice
 - Name: studentmarkservice
 - Description: Microservice For Student Mark Management
 - Package name: com.edu.vnnvjiet.sms
 - Packaging: ☒ **Jar**, ☐ War

Here are the details about the chosen configurations:

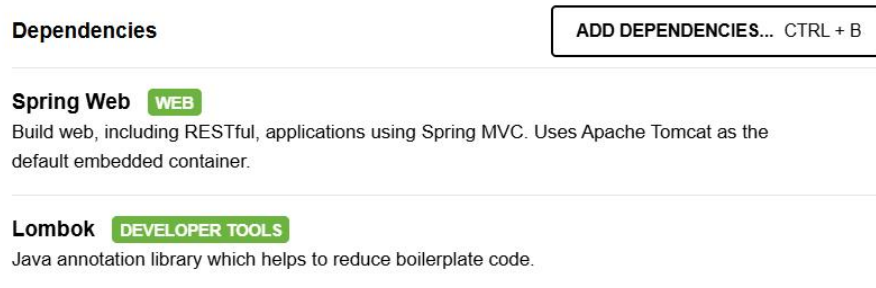
1. **Project : Maven** is chosen as it is the defacto standard for Java Spring Boot based application **Build, Package & Project Management** tool
2. **Spring Boot Version: "3.5.7"** is chosen because SNAPSHOT versions are not stable versions and M2 (Milestone 2) versions are still under testing.
3. **Project Meta Data:**
 - a) **Group:** <Organization Type>.<Organization Short Name>.<Project Short Name>
 - b) **Artifact:** Name of the project output (Name of the jar)
 - c) **Name:** Name of the project

- d) **Description:** About the project in a short sentence
- e) **Package Name:** Root Package Name used in Java application
- f) **Packaging:** Java ARchive File (JAR) is chosen as Web ARchive is used for Enterprise Java Beans (EJB) technology which is a predecessor for Microservices.

Adding Dependency Jars

Owing to the above configuration, the final **Project Template** that will be generated from the **Spring Initializer Web Site** is automatically configured for using all the necessary Jar files for building Spring Boot based application. However, it may be necessary to include configurations in the **Project Template** for **other jars** supported by **Spring Boot Framework** used by application.

This can be done in the same web page in the right hand side panel, by clicking the “**ADD DEPENDENCIES...**” button, searching for following Jars and add them to the list. Here is the final list of Jars files selected on the page:



Here are the details about the Jars that are chosen:

1. **Spring Web:** This is the Jar file that contains Tomcat web server that run RESTful applications that are built using Spring MVC.
2. **Lombok :** This is the Jar file for automatically including boiler plate code for program elements like **classes** or **fields** or methods when they are annotated using Lombok defined **Java Annotations**.

Generating the Project Template

After selecting to the above dependency jar, it is time to generate the **Project Template** that combines all the configurations chosen until this point. This can be done by clicking the “**GENERATE**” push button as follows:



1. The “**GENERATE**” button creates a zip file (in this example it is **studentmarkservice.zip** file) which normally stored in “Downloads” folder
2. The “**EXPLORE**” button lets the user to see how the project template is going to be before clicking the “**GENERATE**” button.
3. The “**...**” button is a utility button for book marking and sharing the template

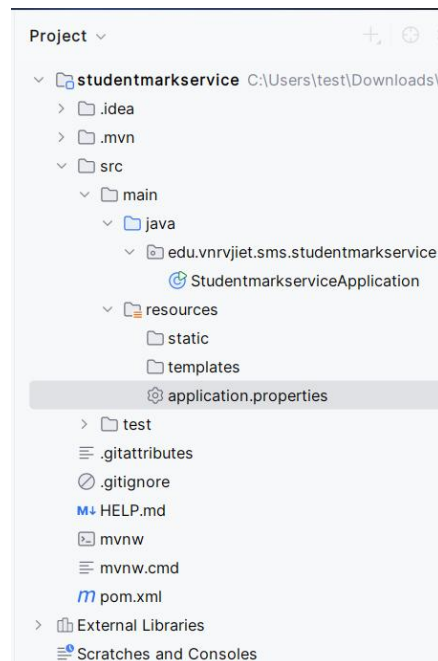
Extracting the Project Template ZIP Locally

Extracting the zip file into a root folder where the project needs to be created would allow **IntelliJ** to open the the extracted **Project Template**.

Step 2: Making The Project Template Ready For Coding Microservices

Understanding Project Template Folder Structure

When **IntelliJ IDE** opens the project folder where **Project Template** zip file was extracted, here is how the project structure is going to look like:



1. **.idea Folder**: Stores project-specific settings and metadata (Details out of scope for this document)
2. **.mvn Folder**: Used for configuring **Maven** build behavior at the project level. (Details out of scope for this document)**src Folder**: Used for storing code used in the project
 - a) **main Sub-Folder**: Used for storing the “Source Code” & “Configurations” used in the project
 - i. **java Sub-Sub-Folder**: Used for storing the “Java Source Code Files” in various packages created as sub folders. **Entire Source code is discussed in other steps.**
 - ii. **resources Sub-Sub-Folder**: Used for storing the different “Configuration” property files such as Application Configuration (**application.properties**), Logging Configurations, Resource Bundles, SQL Scripts, Images (in **static Sub-Sub-Sub folder**) etc. Ignore other sub folders “static”, “template” for now
 - b) **test Sub-Folder**: Used for storing the “Unit Testing Code” & “Configurations” for them
 - i. **java Sub-Sub-Folder**: Used for storing the “Java Unit Test Code Files” in various packages created as sub folders
3. **.gitattributes/.gitignore/HELP.md/mvnw/mvnw.cmd Files**: Details out of scope for this document
4. **pom.xml File**: **Project Object Model** in short **pom** file is the XML file that contains all the configurations made in the **Spring Initializer Web Site** while generating the project template zip file.

Note: Any other IDE that supports Java Spring Booth Project can be used instead of IntelliJ Community Edition.

Understanding Important Content Of pom.xml File

Initial couple of lines of pom.xml are about to XML version and about **Maven** project schema namespace and its location, followed by some important project information given as follows:

Header :

```
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>3.5.7</version>
9          <relativePath/> <!-- lookup parent from repository -->
10     </parent>
11     <groupId>edu.vnrvjiet.sms</groupId>
12     <artifactId>studentmarkservice</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name>studentmarkservice</name>
15     <description>Microservice For Student Mark Management</description>
16     <url/>
17     <licenses...>
20     <developers...>
23     <scm...>
29     <properties>
30         <java.version>21</java.version>
31     </properties>
```

1. **modelVersion Element:** POM file version used by Maven parser for knowing the structure of the file
2. **parent Element:** Contains “**groupId**”, “**artifactId**”, “**version**” child elements that denote Spring Framework details that is selected in [Spring Initializer Web Site](#)
3. “**groupId**”, “**artifactId**”, “**version**” Elements: Denote project specific details configured in [Spring Initializer Web Site](#)
4. “**url**”, “**licenses**”, “**developers**”, “**scm**” Elements: Details out of scope for this document
5. **properties Element:** Contains sub elements such as **java.version**, **encoding**, **junit.version** etc.

Dependencies :

```
32     <dependencies>
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-web</artifactId>
36         </dependency>
37         <dependency>
38             <groupId>org.projectlombok</groupId>
39             <artifactId>lombok</artifactId>
40             <optional>true</optional>
41         </dependency>
42         <dependency>
43             <groupId>org.springframework.boot</groupId>
44             <artifactId>spring-boot-starter-test</artifactId>
45             <scope>test</scope>
46         </dependency>
47         <dependency>
48             <groupId>org.junit.jupiter</groupId>
49             <artifactId>junit-jupiter-api</artifactId>
50             <version>5.11.4</version>
51             <scope>test</scope>
52         </dependency>
53     </dependencies>
```

The **dependencies** element contain all dependency jars included in [Spring Initializer Web Site](#) + any third party jars not set there. Here are those Jars

1. **spring-boot-starter-web.jar**: This Spring Boot's starter dependencies jar contains predefined set of dependencies including **Tomcat Server** bundled together that avoid manual configurations
2. **spring-boot-starter-test.jar**: Spring Boot starter dependency that bundles together the most commonly used testing libraries & frameworks for unit, integration, and Spring Boot-specific testing.
3. **lombok.jar, junit-jupiter-api.jar**: Third party jars used for connecting to PostgreSQL database, for generating boilerplate code and for writing JUnit test cases respectively.

Configure Application Properties

The file "**application.properties**" file (or "application.yml") is used to configure all the properties used by the application. Here is what is used in the project created:

```
1     server.port=8100
2     spring.application.name=studentmarkservice
```

Property Name	Description	Notes
spring.application.name	Name of the Spring Boot Application	N/A
server.port	Tomcat Web Server Port to be used	N/A

Step 3: Add Source Code

StudentMarkServiceApplication Class

This is the “**main**” class used to start the application. Since this application is a Microservice, it should continue to run infinitely until is stopped. The **@SpringBootApplication** annotation on the class adds a hidden infinite loop that listens to all the REST/HTTP requests and channel them to a **Microservice Controller** class object. The **Microservice Controller** class contains the handling for REST methods.

```
1 package edu.vnrvjiet.sms;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication  & Somanadha Satyadev Bulusu *
7 public class StudentMarkServiceApplication {
8
9     public static void main(String[] args) {  & Somanadha Satyadev Bulusu *
10         SpringApplication.run(StudentMarkServiceApplication.class, args);
11         System.out.println("Tomcat running for Student Mark Microservice...");
12     }
13 }
```

Student Class

This is the class represents a student. Here is the code for the class and explanation of important elements of the class

```
1 package edu.vnrvjiet.sms;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5
6 // this is the Domain object
7 @Data 35 usages  & Somanadha Satyadev Bulusu
8 @AllArgsConstructor
9 public class Student {
10     private Integer id;
11     private String name;
12     private Integer maths;
13     private Integer physics;
14     private Integer chemistry;
15 }
```

1. **@Data Annotation On The Class:** This annotation is given by **lombok.jar**. This annotation would inherently adds access (get) & mutate (set) methods for all the fields of the class
2. **@AllArgsConstructor Annotation On The Class:** This annotation is given by **lombok.jar**. This annotation would inherently adds a constructor that takes arguments matching with all private fields of the class

StudentMarkServiceController Class

This is the class has the crux of **REST/HTTP based Microservice** code. This class exposes **POST, GET, DELETE & UPDATE HTTP** methods. Most of the code in the class are strait forward but there are few subtleties in the class worth noting about. They are given after the code:

```
1 package edu.vnrvjiet.sms;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.*;
7
8 import java.util.List;
9
10 @RestController no usages Somanadha Satyadev Bulusu *
11 @RequestMapping("sms")
12 public class StudentMarkServiceController {
13     @Autowired 5 usages
14     private StudentMarkService studentMarkService;
15
16     @PostMapping("addOne") no usages Somanadha Satyadev Bulusu *
17     public ResponseEntity<Student> addOneStudent(@RequestBody Student student) {
18         student = studentMarkService.addOneStudent(student);
19         return new ResponseEntity<>(student, HttpStatus.CREATED);
20     }
21
22     @GetMapping("findAll") no usages Somanadha Satyadev Bulusu *
23     public ResponseEntity<List<Student>> findAllStudents() {
24         List<Student> studentList = studentMarkService.findAllStudents();
25         return new ResponseEntity<>(studentList, HttpStatus.OK);
26     }
27
28     @GetMapping("findById") no usages Somanadha Satyadev Bulusu *
29     public ResponseEntity<Student> findById(Integer id) {
30         Student student = studentMarkService.findById(id);
31         return new ResponseEntity<>(student, HttpStatus.OK);
32     }
33
34     @DeleteMapping("deleteById") no usages Somanadha Satyadev Bulusu *
35     public ResponseEntity<Student> deleteById(Integer id) {
36         Student student = studentMarkService.deleteById(id);
37         return new ResponseEntity<>(student, HttpStatus.OK);
38     }
39
40     @PutMapping("update") no usages Somanadha Satyadev Bulusu *
41     public ResponseEntity<Student> updateStudent(@RequestBody Student student) {
42         student = studentMarkService.updateStudent(student);
43         return new ResponseEntity<>(student, HttpStatus.OK);
44     }
45 }
```

1. **@RestController Annotation on the Class:** This annotation at **line # 10** in the code, identifies this class as Spring MVC **Microservice Controller** that handles web requests. So, in this **Student Mark Service** application case, the class "**StudentMarkServiceController**" becomes the Controller class for all **REST/HTTP Web Requests** coming to the **Microservice**
2. **@RequestMapping("sms") Annotation on the Class:** This annotation at **line # 11** in the code, first maps all **REST/HTTP** requests to the **Microservice Controller** with base URL suffixed with **"/sms"**. In this **Student Mark Service** application case, if the base URL is **"http://localhost:8100"** then in order to access the **Microservice Controller** the URL would be **"http://localhost:8100/sms"**
3. **@Autowired Annotation on the Field:** This annotation at **line # 13** in the code, on the **"StudentMarkService"** field is used for Dependency Injection (DI). This means instead of manually creating the instance with **"new"** operation, Spring framework automatically injects the required object (bean) into a class, . So, in this **Student Mark Service** application case, while an object of the class **"StudentMarkServiceController"** is being instantiated, an object of type **"StudentMarkService"** is also gets created and it is automatically assigned to the **"StudentMarkService"** field.
4. **@PostMapping("addOne") Annotation on the Method:** This annotation at **line # 16** in the code, maps all **HTTP POST** requests from the client to the method **"addOneStudent"** when the Base URL with **Microservice Controller** suffix is further suffixed with **"/addOne"**. So, in this **Student Mark Service** application case, if the URL for **Microservice Controller** is **"http://localhost:8100/sms"**, then to access the **"addOne"** HTTP POST method, URL that needs to be used would be **"http://localhost:8100/sms/addOne"**.
5. **@RequestBody Annotation on the Method Parameter:** This annotation at **line # 17** in the code, converts the **JSON (or XML)** payload given as part of client **Request Message** body, to a **Method Parameter** type object, on which this **@RequestBody** annotation is used. Then that object is passed as an argument to the **Method Parameter**. In this **Student Mark Service** application case, when client indirectly calls the the method **"addOneStudent"** through the **HTTP POST Request Message** with its body containing **JASON** text by hitting the URL **"http://localhost:8100/sms/addOne"**, then that **JASON** text gets converted to **"Student"** object and passed to the **"student"** parameter. So the JASON text should be in the following format:

```
{
  "id": <INTEGER STUDENTID IN QUOTES>,
  "name": <NAME OF THE STUDENTIN QUOTES>,
  "maths": <STUDENT MARKS IN MATHS>,
  "physics": <STUDENT MARKS IN PHYSICS>,
  "chemistry": <STUDENT MARKS IN CHEMISTRY>
}
```

Note: HTTP PUT method works exactly like HTTP POST method, except that it is an UPDATE instead of CREATE

6. **@GetMapping Annotations on the findAll Methods:** The **@GetMapping** annotated method at **line # 38** gets invoked for **HTTP GET** when the client calls the following URL :

a) **"http://localhost:8100/sms/findAll"**

This method would return all the student data that is created and stored through the **StudentMarkService & StudentMarkServiceRepository** classes

7. **@GetMapping, @DeleteMapping Annotations on the findById & deleteById Methods:** These **@GetMapping** and **@DeleteMapping** annotated methods at line # 61 and line # 83 respectively, gets invoked for **HTTP GET & HTTP DELETE** methods respectively. These methods **findById & deleteById** returns and deletes a Student object respectively, when a **student Id** is passed to them as parameter. Main difference difference between these {**HTTP GET, HTTP DELETE**} requests against **HTTP POST** request is that the parameter for these requests don't take any annotation. The Client URL invocation works slightly different to **HTTP POST** method. In this **Student Mark Service** application case, when client indirectly calls:

a) "**findById**" through the **HTTP GET** then the client URL would be "**http://localhost:8100/sms/findById/id=<ID TO BE FOUND>**"

b) "**deleteById**" through the **HTTP DELETE** then the client URL would be "**http://localhost:8100/sms/deleteById/id=<ID TO BE DELETED>**"

StudentMarkService Class

This is the class that contain all the **Business Logic** implemented. Methods from the **Microservice Controller (StudentMarkServiceController)** class object calls into these methods, as an instance of this class is auto wired into the **Microservice Controller** class.

Since this is a very simple application, there is no business logic that is implemented in this class. Only thing that each method in this class does, is that it delegates the call to an instance of **StudentMarkServiceRepository** class, which is **auto wired** through dependency injection at line #10 & line #11.

```
1 package edu.vnrvjiet.sms;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5
6 import java.util.List;
7
8 @Service 3 usages Somanadha Satyadev Bulusu
9 public class StudentMarkService {
10     @Autowired 5 usages
11     StudentMarkServiceRepository studentMarkServiceRepository;
12
13     public Student addOneStudent(Student student) { 2 usages Somanadha Satyadev Bulusu
14         return studentMarkServiceRepository.add(student);
15     }
16
17     public List<Student> findAllStudents() { 1 usage Somanadha Satyadev Bulusu
18         return studentMarkServiceRepository.findAll();
19     }
20
21     public Student findById(Integer id) { 1 usage Somanadha Satyadev Bulusu
22         return studentMarkServiceRepository.findById(id);
23     }
24
25     public Student deleteById(Integer id) { 1 usage Somanadha Satyadev Bulusu
26         return studentMarkServiceRepository.deleteById(id);
27     }
28
29     public Student updateStudent(Student student) { 1 usage Somanadha Satyadev Bulusu
30         return studentMarkServiceRepository.update(student);
31     }
32 }
```


StudentMarkServiceRepository Class

StudentMarkServiceRepository is the class that is annotated with **@Repository**, but it doesn't provide any special functionality, except that it indicates that this is a **Data Access Layer** class.

```
1 package edu.vnrvjiet.sms;
2
3 import org.springframework.stereotype.Repository;
4
5 import java.util.ArrayList;
6 import java.util.HashMap;
7 import java.util.List;
8
9 @Repository 2 usages & Somanadha Satyadev Bukuru
10 public class StudentMarkServiceRepository {
11     HashMap<Integer, Student> idAndStudentHashMap = new HashMap<>(); 10 usages
12
13     public Student add(Student student) { 1 usage & Somanadha Satyadev Bukuru
14         if (student != null && !idAndStudentHashMap.containsKey(student.getId())){
15             idAndStudentHashMap.put(student.getId(), student);
16         }
17         return student;
18     }
19
20     public List<Student> findAll() { 1 usage & Somanadha Satyadev Bukuru
21         return new ArrayList<>(idAndStudentHashMap.values());
22     }
23
24     public Student findById(Integer id) { 2 usages & Somanadha Satyadev Bukuru
25         Student student = null;
26         if (id != null && idAndStudentHashMap.containsKey(id)){
27             student = idAndStudentHashMap.get(id);
28         }
29         return student;
30     }
31
32     public Student deleteById(Integer id) { 1 usage & Somanadha Satyadev Bukuru
33         Student student = null;
34         if (id != null && idAndStudentHashMap.containsKey(id)){
35             student = idAndStudentHashMap.get(id);
36             idAndStudentHashMap.remove(id);
37         }
38         return student;
39     }
40
41     public Student update(Student student) { 1 usage & Somanadha Satyadev Bukuru
42         Student existingStudent = null;
43         if (student != null && idAndStudentHashMap.containsKey(student.getId())){
44             existingStudent = idAndStudentHashMap.get(student.getId());
45             existingStudent.setName(student.getName());
46             existingStudent.setMaths(student.getMaths());
47             existingStudent.setPhysics(student.getPhysics());
48             existingStudent.setChemistry(student.getChemistry());
49         }
50         return existingStudent;
51     }
52 }
```

The **StudentMarkServiceRepository** class doesn't perform any database access even though it is annotated as Repository, rather it uses a **HashMap** to manage student instances based on the CRUD method (For Create - "add" method, for Retrieve - "findAll" & findById" methods, for Update - "update" method, for Delete - "deleteById") called on it.

Step 4: Running DevOps process

DevOps Process

As coding is completed for “**Student Mark Service**” application, next step is to complete DevOps process so that application is ready to use without much manual intervention. Here are those DevOps Process Steps

1. Store the project in local git repository
2. Start Jenkins Servier (Assuming it is running on port 8080)
3. Run ngrok with the following command “ngrok http 8080”
4. Create the placeholder for the project in GitHub
5. Create a web-hook in the project for Jenkins to be notified using ngrok
6. Start Jenkins
7. Create Build Pipeline in Jenkins for building the project automatically whenever web-hook is called.
8. Push the project stored in the local git to GitHub

From the above steps, last step should Build the project in Jenkins pipeline to get a JAR file, then Create a docker image with that JAR and finally using Kubernetes (K8s) Deployment, Service & Namespace manifestation files create Pod(s) and host in Docker for desktop (Windows). Once it is done using postman the microservice is tested.

For building in Jenkins, Dockerizing it and for Orchestrating through Kubernetes (K8s), 5 scripts are needed. They are

1. Jenkinsfile

2. Dockerfile

3. Namespace-template

4. Deployment-template

5. Service-template

Jenkinsfile and **Dockerfile** are present in the project root directory but remaining template files are in **k8s** sub directory.

```

1 pipeline {
2     agent any
3     environment {
4         APP_NAME = "studentmarkservice"
5         APP_NAMESPACE = "${APP_NAME}-ns"
6         IMAGE_NAME = "${APP_NAME}-image"
7         IMAGE_TAG = "${BUILD_NUMBER}-${BUILD_NUMBER}"
8         APP_PORT = 8100
9         NODE_PORT = 30081
10        REPLICAS_COUNT = 2
11    }
12    stages {
13        stage('Checkout') {
14            steps {
15                git branch: 'main', url: 'https://github.com/somanadha/studentmarkservice.git'
16            }
17        }
18        stage('Build') {
19            steps {
20                // Build docker image using Dockerfile
21                bat "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} -f Dockerfile ."
22            }
23        }
24        /*
25        stage('Run Container') {
26            steps {
27                script {
28                    // Run docker container instance
29                    bat "docker run -d --name ${APP_NAME} -p 8100:8100 ${IMAGE_NAME}:${IMAGE_TAG}"
30                }
31            }
32        }
33        */
34        stage('K8s Container Deployment') {
35            steps {
36                script {
37                    withEnv(["KUBECONFIG=c:\\users\\test\\.kube\\config"]) {
38                        // Replace variables in YAML and create concrete files:
39                        bat "envsubst < k8s/namespace-template.yaml > k8s/namespace.yaml"
40                        bat "envsubst < k8s/deployment-template.yaml > k8s/deployment.yaml"
41                        bat "envsubst < k8s/service-template.yaml > k8s/service.yaml"
42
43                        // Apply manifests
44                        bat "kubectl apply -f k8s/namespace.yaml --validate=false"
45                        bat "kubectl apply -f k8s/deployment.yaml --validate=false"
46                        bat "kubectl apply -f k8s/service.yaml --validate=false"
47                    }
48                }
49            }
50        }
51    }
52    post {
53        success {
54            echo "✅ Checkout, Build, Dockerize & Deploy completed successfully!"
55        }
56        failure {
57            echo "❌ Build failed!"
58        }
59    }
60 }

```

1. **"pipeline"** : This directive at **line # 1** indicates that this is a Jenkins Pipeline script file
2. **"agent any"** : This directive at **line # 2** indicates that the build pipeline (or stage) can run on any available Jenkins agent/node
3. **"environment"**: This directive at **line # 3** is to define environment variables that will be available to the pipeline or to specific stages.
4. **"stages"**: This directive at **line # 11** is used to define the sequence of tasks (stages) that the pipeline will execute. Here are the stages used in this application:

- a) **"Checkout"**: Stage defined at **line # 12** is for getting (checkout) the code from the **Project** pushed into **GitHub Branch** in the step 5 of DevOps process.
- b) **"Build"**: Stage defined at **line # 18** is for building the Jar and creating a docker image (Dockerization) out of the jar using the script file named **Dockerfile**
- c) **"Run Container"**: Stage defined at **line # 24** is for creating a Docker Container out of the image and running it on port **8100:8100 (Host:Container)** is commented because instead of directly creating container instances, K8s Pods are created.
- d) **"K8s Container Deployment"**: Stage defined at **line # 34** is for creating namespace, deployment & service in Docker Desktop.
 - i. **"withEnv"** : This is a directive that allows define/override environment variables dynamically at runtime. In the current context it overrides KUBECONFIG so that it looks for configuration file at the specific path when connecting to Kubernetes cluster.
 - ii. **"bat"**: Is to indicate that run command followed by it in a windows command window
 - iii. **"envsubst"**: Is a a Linux command-line utility that performs environment variable substitution.It dynamically injects values into YAML templates before applying them with kubectl. In the current context this command takes a configuration YAML template file, that uses ENVIRONMENT variables defined in the **"environment"** section of this Jenkinsfile, replaces those variables with the values defined and then copies the new content (variable replaced with values) into a new file. It uses **"<"** for taking a file as input and **">"** for writing the new content as output

In the current scenario contents from 3 templates files in the "k8s" subdirectory namely **"namespace-template.yaml"**, **"deployment-template.yaml"** & **"service-template.yaml"** are copied into 3 new files (after their content goes through environment variable substitution) in to the same "k8s" subdirectory with names **"namespace.yaml"**, **"deployment.yaml"** & **"service.yaml"** respectively.

- iv. **"kubectl"**: Instructs Kubernetes to apply the configuration read from a file given after **"-f"** option and create resources if they don't exist, or update it if they already exist .

In the current scenario there are 3 files in the "k8s" subdirectory namely **"namespace.yaml"**, **"deployment.yaml"** & **"service.yaml"** are read and K8s resources namely a Namespace, Pod(s), a Deployment instance & a Service instance are created

- v. **"post"** : Actions to run after pipeline/stage completed (always, success, failure).


```

1  # Stage 1: Build the application and resolve dependencies
2  FROM maven:3.9.4-eclipse-temurin-21-alpine AS build
3  WORKDIR /app
4  COPY pom.xml .
5  COPY src ./src
6
7  # Download dependencies only to cache them for later build stages
8  # Download dependencies to the target/dependency directory for later build stages
9  RUN mvn dependency:copy-dependencies
10
11 # Package the application as a fat jar or a standard jar
12 RUN mvn -X package -DskipTests
13
14 # Stage 2: Create the final image
15 FROM eclipse-temurin:21-alpine
16 WORKDIR /app
17
18 # Copy the application JAR (LEAN/THIN) & its dependencies from the build stage to make a FAT/UBER JAR
19 COPY --from=build /app/target/*.jar studentmarkservice.jar
20
21 # Application is running in port 8100
22 EXPOSE 8100
23
24 # Command to run the application with all dependencies on the classpath
25 #CMD ["java", "-cp", "studentmarkservice.jar:dependency/*", "edu.vnrvjiet.sms.StudentMarkService"]
26 ENTRYPOINT ["java", "-jar", "studentmarkservice.jar"]

```

It is evident from the above file that there are **2 stages** in the **Dockerization** process. In **stage 1**, the **studentmarkservice.jar** JAR file is built using **Maven** (uses **pom.xml** for dependency and package management) tool. So the base image for the **stage 1** requires **Maven** & **JDK**. Here are the command that run in **stage 1**:

1. **"FROM maven:3.9.4-eclipse-temurin-21-alpine AS build"** : This command at **line # 2** is to select the **maven:3.9.4-eclipse-temurin-21-alpine** as the **Base Image**. This image is the starting point for the **Student Mark Service** application's **Build** process. The **Base Image** that is selected in the instruction has **Maven 3.9.4** running on **JDK 21** (Not JRE). The main reason for choosing this **Base Image** is to leverage the existing well-maintained upstream image from [Docker Hub](#) that contains what is required for build process, instead of building everything from scratch. The **Base Image** is also given an alias name (**AS**) as **build**.
2. **"WORKDIR /app"** : This command at **line # 3** is to set the working directory as **" /app "** inside the **Docker Image** for any subsequent commands. This avoids repeating long paths in the subsequent commands.
3. **"COPY pom.xml & COPY src ./src"**: These commands at **line # 4 & line # 5** are given to copy **pom.xml** file and **"./src"** directory respectively from local build context (host machine) into the **Docker Image** file system. So command at line # 4 copies **pom.xml** in the host machine to **" /app "** folder of the **Docker Image** and command at line # 5 copies **"src"** folder and its sub folders to **" /app/src "** folder maintaining the same hierarchy.
4. **"RUN mvn dependency:copy-dependencies"**: This command at **line # 9** executes the **Maven Dependency Plugin** that copies all external dependencies (**JARS** specified in **pom.xml** file) into **"app/target/dependency/"** folder in the **Docker Image**.

5. **"RUN mvn -X package -DskipTests"**: This command at line # 12 runs the Maven build lifecycle up to the **package** phase, which builds the **studentmarkservice.jar (LEAN/THIN) JAR**. The **"-X"** option enables debug logging, helpful for diagnosing build issues and the option **"-DskipTests"** compiles tests but skips running them, which reduces build time inside docker image.

Note: LEAN/THIN JAR means that the JAR contains only the compiled binaries (class files) from the application code only, not from the dependency JARs.

6. Once the **studentmarkservice.jar JAR** is built in **phase 1, Maven & JDK** are no longer required and only components required are: the **studentmarkservice.jar JAR, its dependencies & JRE 21 (Not JDK)**. So, in the **stage 2**, another **Base Image** is chosen, which contains **JRE 21** only. The **studentmarkservice.jar JAR** and its dependency **JARs** built from **stage 1** are copied on top of this new **Base Image** making the final **Docker image** more compact. Here are the command that run in **stage 2**:

1. **"FROM eclipse-temurin:21-alpine"** : This command at line # 15 is to select the **eclipse-temurin:21-alpine** as the new **Base Image**. It has **JRE 21** only.

2. **"WORKDIR /app"** : This command at line #16 is to set the working directory as **" /app"** inside the new **Docker Image** for any subsequent commands.

3. **"COPY --from=build /app/target/*.jar studentmarkservice.jar"** : This command at line #19 takes the **studentmarkservice.jar (LEAN/THIN) JAR** file and its dependencies (**--from**) from the **build** image (alias name for the **Docker Image** in **stage 1**) and then bundles all of them into this **Base Image** of **stage 2** as one single file, that is **studentmarkservice.jar (FAT/UBER) JAR** file.

4. **"EXPOSE 8100"** : This command at line #22 declares that the container intends to listen on port 8100 at runtime. It's just a documentation/metadata instruction in the **Dockerization** process. It does not actually publish the port to the host machine

5. **"ENTRYPOINT [\"java\", \"-jar\", \"studentmarkservice.jar\"]"** : This command at line # 26 instructs docker image to run **"java -jar studentmarkservice.jar"** command when a docker container (created from the docker image) starts

namespace-template.yaml

```
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: ${APP_NAMESPACE}
```

When *namespace-template.yaml* goes through environment variable substitution because of following command:

envsubst < k8s/namespace-template.yaml > k8s/namespace.yaml

Environment variable defined in the Jenkinsfile :

```
3  environment {
4      APP_NAME = "studentmarkservice"
5      APP_NAMESPACE = "${APP_NAME}-ns"
6      IMAGE_NAME = "${APP_NAME}-image"
7      IMAGE_TAG = "${BUILD_NUMBER}${BUILD_NUMBER}"
8      APP_PORT = 8100
9      NODE_PORT = 30081
10     REPLICA_COUNT = 2
11 }
```

Output file would be *Namespace.yaml* with content:

```
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: studentmarkservice-ns
```

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: ${APP_NAME}-deployment
5    namespace: ${APP_NAMESPACE}
6  spec:
7    replicas: ${REPLICA_COUNT}
8    selector:
9      matchLabels:
10       app: ${APP_NAME}
11    template:
12      metadata:
13        labels:
14          app: ${APP_NAME}
15      spec:
16        containers:
17          - name: ${APP_NAME}-container
18            image: ${IMAGE_NAME}:${IMAGE_TAG}
19            imagePullPolicy: IfNotPresent
20            ports:
21              - containerPort: ${APP_PORT}
```

When *deployment-template.yaml* goes through environment variable substitution because of following command:

envsubst < k8s/deployment-template.yaml > k8s/deployment.yaml

Environment Variable defined in the Jenkinsfile :

```
3      environment {
4        APP_NAME = "studentmarkservice"
5        APP_NAMESPACE = "${APP_NAME}-ns"
6        IMAGE_NAME = "${APP_NAME}-image"
7        IMAGE_TAG = "${BUILD_NUMBER}${BUILD_NUMBER}"
8        APP_PORT = 8100
9        NODE_PORT = 30081
10       REPLICA_COUNT = 2
11     }
```

Output file would be *deployment.yaml* with content:

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: studentmarkservice-deployment
5    namespace: studentmarkservice-ns
6  spec:
7    replicas: 2
8    selector:
9      matchLabels:
10       app: studentmarkservice
11  template:
12    metadata:
13      labels:
14       app: studentmarkservice
15    spec:
16      containers:
17       - name: studentmarkservice-container
18         image: studentmarkservice-image:5656
19         imagePullPolicy: IfNotPresent
20         ports:
21         - containerPort: 8100
```


service-template.yaml

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: ${APP_NAME}-service
5    namespace: ${APP_NAMESPACE}
6  spec:
7    type: NodePort
8    selector:
9      app: ${APP_NAME}
10   ports:
11     - port: ${APP_PORT}          # Service port (inside cluster)
12       targetPort: ${APP_PORT}    # Container port
13       nodePort: ${NODE_PORT}     # Port accessible from Postman (your host)
```

When *service-template.yaml* goes through environment variable substitution because of following command:

envsubst < k8s/service-template.yaml > k8s/service.yaml

Environment variable defined in the Jenkinsfile :

```
3      environment {
4        APP_NAME = "studentmarkservice"
5        APP_NAMESPACE = "${APP_NAME}-ns"
6        IMAGE_NAME = "${APP_NAME}-image"
7        IMAGE_TAG = "${BUILD_NUMBER}-${BUILD_NUMBER}"
8        APP_PORT = 8100
9        NODE_PORT = 30081
10       REPLICAS_COUNT = 2
11     }
```

Output file would be *Namespace.yaml* with content:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: studentmarkservice-service
5    namespace: studentmarkservice-ns
6  spec:
7    type: NodePort
8    selector:
9      app: studentmarkservice
10   ports:
11     - port: 8100          # Service port (inside cluster)
12       targetPort: 8100    # Container port
13       nodePort: 30081     # Port accessible from Postman (your host)
```